Section 5 — Evaluation

Offline metrics, online A/B testing, and debiasing — with all worked examples, formulas, and clarifications. Companion to Sections 1–4.5.

Contents

The organizing idea: each layer exists because the previous one can lie. Evaluation is three layers of increasing cost and trustworthiness. Offline metrics are cheap but measured on biased logged data. Online A/B tests are trustworthy but slow and risky. Debiasing tries to make the cheap offline estimates trustworthy enough to predict the expensive online result. The whole section is about that tension.

Part A — Offline evaluation

Why offline first. Before exposing anything to users, you score the model on logged historical data you already have. It's cheap, instant, repeatable, and risk-free — it's how you iterate and kill bad models before they touch traffic.

Two framing points the interview rewards

1. You evaluate each stage of the funnel differently, not the whole system with one number. The candidate generators, the light ranker, and the heavy ranker have different jobs, so they get different metrics. A candidate generator's job is *coverage* — don't lose good videos — so it's judged on recall. The heavy ranker's job is the final *order* of the ~100 survivors, so it's judged on top-heavy ranking quality (NDCG). Using NDCG to grade a candidate generator, or recall@10k to grade the heavy ranker, is a category error.

2. You evaluate each prediction head separately. The heavy ranker emits several probabilities — P(click), P(like), P(complete), P(return) — and each is a small classifier you can score on its own before they're combined into the value model.

So offline evaluation splits into two questions: are the per-head *probabilities* accurate, and is the final *ordering* good?

A1 · Per-head classification quality

Sourcing: the principle of separate per-head evaluation is Hello Interview; the Reels UTIS work reports precision/recall/accuracy explicitly. The specific curves below (ROC-AUC, PR-AUC, calibration) are standard-ML knowledge supplied to flesh that out — they are not named in the sources.

The building blocks: the four bins and the four rates

Pick any threshold for "flag as likely click." Every (shown) video then falls into one of four bins:

bin	meaning	example values
TP (true positive)	clicked, and model flagged it	8
FP (false positive)	not clicked, but model flagged it	52
FN (false negative)	clicked, but model missed it	2
TN (true negative)	not clicked, model didn't flag	938

> **Recall (= TPR)** = TP / (TP + FN) — of the videos that *were* clicked, how many did the model catch?
> **FPR** = FP / (FP + TN) — of the videos *not* clicked, how many were wrongly flagged?
> **Precision** = TP / (TP + FP) — of the videos the model *flagged*, how many were actually clicked?

Recall and precision share the same numerator (TP) but differ in denominator: recall divides by all actual positives, precision by all flagged. That denominator difference is the entire ROC-vs-PR story below.

Clarification — what is an "impression"?

An impression = one video *shown to a user* and counted as seen — a (user, video, time) exposure event. Not the catalog, not unique videos: if a feed shows you 60 videos that's 60 impressions; the same video shown to 1,000 users is 1,000 impressions. "Shown" is distinct from "clicked" — which is exactly why positives are rare: you score everything you put in front of people, and people engage with a tiny slice. CTR is literally clicks / impressions, and the click head predicts an outcome *conditional on the video being an impression*.

ROC-AUC — the workhorse, and what "AUC" usually means

"AUC" by itself is ambiguous: it's the area under *a* curve, and there are two curves in play — the ROC curve (TPR vs FPR) and the Precision–Recall curve. When someone says "AUC" with no qualifier they almost always mean **ROC-AUC**. Interpretation: if I pick one video the user actually clicked and one they didn't, how often does the model score the clicked one higher? 1.0 = always, 0.5 = coin-flip. Threshold-independent, which is why it's the default headline number for a CTR head.

WORKED EXAMPLE — ROC-AUC BY PAIRWISE COUNTING

Five candidates, user clicked two (✓), with model P(click):

```
video   P(click)   clicked?
A        0.9         ✓
B        0.8         ✗
C        0.6         ✓
D        0.4         ✗
E        0.2         ✗
```

Check every (clicked, not-clicked) pair — is the clicked one scored higher? (A>B)✓ (A>D)✓ (A>E)✓ (C>B)✗ since 0.6<0.8, (C>D)✓ (C>E)✓. That's 5 of 6 → **ROC-AUC ≈ 0.83**.

The AUC formulas (whole curve, not one threshold)

The four rates above describe *one* threshold. Sweep the threshold from high to low and each setting gives one (FPR, TPR) point and one (Recall, Precision) point — tracing out the two curves. AUC is the area under each:

> **ROC-AUC** = ∫ TPR d(FPR) — computed as the trapezoid sum Σ (ΔFPR)·(avg TPR), or equivalently the *pairwise form*: ROC-AUC = P(random positive scores above random negative) = (correct pairs) / (|P|·|N|)
>
> **PR-AUC** = ∫ Precision d(Recall) — computed as Average Precision: AP = Σ_k ($\text{Recall}_k - \text{Recall}_{k-1}$) · Precision_k

The pairwise form is why the A–E example *is* the ROC-AUC: count pairs where the positive outscores the negative, divide by total pairs (verified: the pairwise count and the trapezoid integral both give 0.8333 on that example). In practice you call `sklearn.metrics.roc_auc_score` / `average_precision_score`; what you say in an interview is the per-threshold formulas, that AUC sweeps all thresholds into one number, and the pairwise intuition.

PR-AUC — the other AUC, for when positives are rare

You need it because **ROC-AUC goes blind when positives are rare** — and PR-AUC is the one that still sees clearly.

Clarification — what does "positives are rare" mean?

A *positive* is the outcome the head predicts — a click for the click head, a like for the like head. *Rare* means it almost never happens: of 1,000 videos shown, maybe 10 get clicked, so the data is ~99% negatives and ~1% positives. That lopsidedness is **class imbalance**, and it's the default for engagement signals (likes are rarer still). It's exactly this rarity that breaks ROC-AUC. If classes were balanced 50/50, FPR's denominator and precision's denominator would be comparable, the two metrics would largely agree, and ROC-AUC would be fine — imbalance is specifically what splits them.

WORKED EXAMPLE — SAME MODEL, SAME THRESHOLD, OPPOSITE VERDICTS

1,000 impressions, only 10 clicks. The model flags 60 videos as likely clicks; 8 are real, 52 are wrong.

- Recall = 8/10 = **0.80** (caught most real clicks)
- FPR = 52/990 = **0.05**
- Precision = 8/60 = **0.13** (87% of the flagged slate is junk)

ROC reads this point as recall 0.80 at FPR 0.05 — *looks excellent*. PR reads the identical point as recall 0.80 at precision 0.13 — *correctly terrible*. The difference is the denominator: FPR divides the 52 mistakes by 990 negatives (they vanish); precision divides them by the 60 flagged (they dominate). ROC is congratulating the model for being right about the 990 easy "no click" cases — never the hard part.

Clarification — if PR says "terrible," why use it?

Because PR isn't the thing that's terrible — **the model is**, and PR is the honest metric that reports it. Serve those 60 flagged videos and 87% is garbage; "precision 0.13" is just the number that says so. ROC's cheerful 0.05 FPR is the *misleading* read — like a thermometer reading 98°F on a feverish patient. You use PR precisely because it catches a failure ROC hides: a metric that stays cheerful while you ship an 87%-wrong slate is the useless one. ROC-AUC still has its place — comparing models on balanced data, or when the positive rate drifts between training and serving — but for rare-positive heads like clicks and likes, PR tells the truth.

Clarification — who decides the ranking PR-AUC is computed on?

The model does: the ranking is just every impression *sorted by the head's predicted score*, highest first. No human picks it. The labels (clicked / not) come from users, recorded in logs after the fact. PR-AUC then asks: did the model's ranking put the users' actual clicks near the top? In a real evaluation you read the click positions straight off the sorted held-out logs.

WORKED EXAMPLE — COMPUTING PR-AUC BY WALKING THE LIST

One threshold gives one PR point; the area needs the full ranking. Suppose the model's 10 true clicks land at sorted ranks **1, 2, 3, 5, 8, 12, 20, 35, 60, 400** (out of 1,000). Walk down the list; at each real click compute precision = clicks-found / rank:

```
rank   1     2     3     5     8     12    20    35    60    400
prec  1.00  1.00  1.00  0.80  0.63  0.50  0.35  0.23  0.15  0.025
```

PR-AUC (Average Precision) = mean of those ten = **0.57**. The same ranking scores **ROC-AUC = 0.95** — superb-looking, because the model is great at the easy job (keeping 990 non-clicks low) while the hard job (all 10 clicks up top) is only half done; the click buried at rank 400 (precision 0.025) drags AP down and ROC barely notices. Method in one sentence: *walk the ranked list top-down, take precision at each real positive, average them.*

Clarification — why is a random model's PR-AUC = the positive rate? Is it the same for all random models?

A random ranking is a shuffle, so *any* top-K chunk of it is a random sample of the population — and a random sample is ~1% positive at any size. So precision sits at ~0.01 at every depth, the PR curve is a flat line at height 0.01, and the area = 0.01 × 1 = **0.01**. All skill-less models on this dataset land there (verified across seeds: 0.011, 0.010, 0.017, 0.008, 0.009 — noise around 0.01). But the floor *moves with prevalence*: likes at 0.2% → floor 0.002; a balanced 50/50 task → floor 0.50. Meanwhile random ROC-AUC is **always 0.5** on any dataset. That asymmetry is why you read PR-AUC against the positive rate, never against 0.5 — your model's 0.57 here beats a 0.01 floor (57× chance); the same 0.57 on a balanced task would barely beat the 0.50 floor.

Which to use — both, with different jobs. ROC-AUC to compare models (threshold-independent, prevalence-independent); PR-AUC as the imbalance reality check. Reporting both is exactly why the Reels UTIS work led with precision *and* recall — its heuristic baseline sat at 48.3% precision / 45.4% recall, numbers raw accuracy would have hidden.

Calibration — separate from ROC-AUC, and easy to forget

Does a predicted 0.3 actually mean a 30% chance? It matters *here specifically* because the value model adds head probabilities — VM = W_{click}·P(click) + W_{like}·P(like) − … — and if they're not on a true probability scale the weighted sum is garbage. The subtlety: ranking by a *single* head survives any monotonic distortion, so calibration is dispensable for one head alone. It becomes essential precisely because you're *combining heads* — a miscalibrated P(like) and a calibrated P(click) don't add to anything meaningful.

DIAGNOSING MISCALIBRATION

Take every impression where the model predicted P(click) ≈ 0.3. If calibrated, ~30% were actually clicked. If only 10% were, the model is overconfident — its "0.3" is really 0.1. A model can post great ROC-AUC and terrible calibration: score every clicked item just barely above every non-clicked one and the ordering is perfect (high AUC) but the probability values are meaningless.

How you fix calibration

Don't retrain. Take the frozen heavy ranker, run it over a held-out day of logged impressions, compare each head's predictions to what actually happened, and learn a small correction. Every method works on the **logit** $z = \log(p/(1-p))$ — the raw score before the sigmoid — because bending it there keeps the output a valid probability and never reorders anything. (σ is the sigmoid, $\sigma(z) = 1/(1+e^{-z})$, the inverse of the logit.)

Clarification — the temperature-scaling mechanics, slowly

A neural head doesn't directly output a probability: its last layer outputs a **logit** $z \in (-\infty, \infty)$, and the sigmoid maps it to a probability. Pipeline: $z \to \sigma(z) \to p$. Reference points: $\sigma(0)=0.5$, $\sigma(1.1)\approx 0.75$, $\sigma(2.2)\approx 0.90$. Suppose the head says $p = 0.9$ but videos scored 0.9 really get clicked only ~75% of the time. Fix in three steps: **(1)** recover the logit: $z = \log(0.9/0.1) = \log 9 \approx 2.2$. **(2)** shrink it by the temperature: $z/T = 2.2/2 = 1.1$ (dividing by $T > 1$ pulls logits toward 0, i.e. probabilities toward 0.5 — "deflating" overconfidence). **(3)** map back: $\sigma(1.1) \approx$ **0.75**. One line: $p_{cal} = \sigma(z/T)$. And because the *same* T divides every logit from that head, the order never changes — if $z_A > z_B$ then $z_A/2 > z_B/2$ — so ROC-AUC and NDCG are untouched; only the probability values get more honest.

Clarification — how is T learned?

By minimizing a loss over a held-out set — the same cross-entropy / negative log-likelihood the head trained on, re-optimized over the single parameter T with everything else frozen: $L(T) = -(1/N)\ \Sigma\ [y_i \log \sigma(z_i/T) + (1-y_i) \log(1-\sigma(z_i/T))]$. T too small → confident-but-wrong predictions punished savagely; T too large → everything collapses toward 0.5 and correct confidence stops being rewarded; the minimum sits where probabilities are honest. It's 1-D, so gradient descent or a plain grid sweep both work in seconds. Verified on a toy overconfident head (logits ±2.2, outcomes only half confirming): loss at T=1 was 0.545, minimized at **T = 1.5** (0.501), which maps a raw 0.9 prediction to 0.81. Held-out data is non-negotiable — fit T on training data and you just relearn the frequencies the model already overfit.

method	formula	knobs	when
Temperature scaling	$p_{cal} = \sigma(z / T)$	1 (T)	Default. Worked: p=0.9, z≈2.2, T=2 → σ(1.1)=0.75.
Platt scaling	$p_{cal} = \sigma(a \cdot z + b)$	2 (a, b)	When a head is also *shifted* (biased high everywhere, e.g. sparse P(like)). Worked: a=0.5, b=−0.2 on z=2.2 → σ(0.9)=0.71.
Isotonic regression	bucket → remap to observed rate	free-form monotone	Most powerful (any monotonic distortion: the bucket predicting 0.30 that really clicks 0.10 becomes 0.30 → 0.10), but data-hungry; needs a large held-out set.

How you know it worked: plot predicted-vs-actual in buckets — a *reliability diagram*; calibrated points sit on the diagonal — and collapse the gap into one number, *Expected Calibration Error* (bucket-size-weighted average distance from the diagonal).

Always measured on held-out data.

A2 · Ranking quality — is the final ordering good?

This is what actually matters: the user sees an order, not probabilities.

NDCG — the metric for graded relevance

The idea. Each video has a true relevance (how much the user would like it). NDCG rewards putting high-relevance videos near the top, and discounts videos placed lower — a great video buried at the bottom barely counts. It earns its keep when relevance has *grades* (e.g. watch-time buckets, or the UTIS 1–5 survey scale).

> **DCG** = Σ relevance / $\log_2$(position + 1) · **NDCG** = DCG / IDCG (ideal order's DCG), rescaled 0–1.
> Fixed discounts: pos 1 → $\log_2 2 = 1.00$ · pos 2 → $\log_2 3 = 1.585$ · pos 3 → $\log_2 4 = 2.00$ · pos 4 → $\log_2 5 = 2.322$

WORKED EXAMPLE — NDCG, VERIFIED

Four videos with true relevance 3, 2, 0, 1 (3 = loved, 0 = ignored). Your ranker shows them in that order: [3, 2, 0, 1].

pos	relevance	discount	term	ideal rel.	ideal term
1	3	1.00	3.00	3	3.00
2	2	1.585	1.26	2	1.26
3	0	2.00	0.00	1	0.50
4	1	2.322	0.43	0	0.00

DCG = 3.00 + 1.26 + 0 + 0.43 = **4.69**. IDCG (best order [3,2,1,0]) = 3.00 + 1.26 + 0.50 + 0 = **4.76**. NDCG = 4.69 / 4.76 = **0.985**.

Reading it: near-perfect. The only mistake — the relevance-1 video below the relevance-0 video — sits at positions 3–4 where the discounts are large, so it costs almost nothing. A swap that high-up would crater the score; buried at the bottom it barely registers. That's the entire point of the log discount.

Gotcha: a second convention exponentiates relevance — term = $(2^{rel} - 1)/\log_2(pos+1)$ — punishing top mistakes harder. The linear form above is the textbook one; just recognize the variant. (Earlier drafts printed the sums as 4.70/4.77 from rounded intermediates; the exact values are 4.69/4.76 — final NDCG ≈ 0.985 either way.)

mAP — the metric for binary relevance

Clarification — what does mAP stand for?

mean Average Precision — two different averaging steps, read inside-out. **AP** (Average Precision) averages over *positions within one user's list*: walk the ranked list, take precision at each relevant item, average. **m** (mean) then averages those APs *across users*. The lowercase "m" is conventional, partly to avoid clashing with MAP = maximum a posteriori.

NDCG needs grades; when relevance is just yes/no, use mAP — which is why Alex Xu reaches for it here (this system's labels are often binary: "liked it / watched half").

WORKED EXAMPLE — AP

Relevant items land at ranks 1, 3, 4. Precision at each hit: rank 1 → 1/1 = 1.00; rank 3 → 2/3 = 0.67; rank 4 → 3/4 = 0.75. AP = (1.00 + 0.67 + 0.75)/3 = **0.81**. mAP = this, averaged over all users.

Recall@K / Precision@K

Two snapshots of the top K: **Precision@K** — of the K shown, how many relevant; **Recall@K** — of all relevant items in existence, how many made the top K.

WORKED EXAMPLE

10 relevant videos exist; your top-20 contains 6. Precision@20 = 6/20 = **0.30**; Recall@20 = 6/10 = **0.60**. Recall@K is the **candidate-generation** metric (coverage); Precision@K and NDCG matter at the **heavy ranker**, where the top slots are what the user sees — the per-stage point made concrete.

Diversity

A list can ace NDCG and precision yet still feel bad if all ten videos are near-identical. Diversity measures how dissimilar the items are *to each other*: take the average pairwise similarity (cosine) of the videos in the list — **low average = diverse**.

Clarification — what are the video vectors? Isn't the model's output probabilities?

Both exist; they're different outputs used for different jobs. The **heavy ranker** outputs probabilities — used to score and order. The **encoders/towers** output **embeddings** — vectors describing the video, written as tuples like (0.6, 0.8): each slot is a learned content dimension and the number is how much of it the video has. Probabilities can't answer "are these two videos similar?" — P(click)=0.3 for both tells you they're equally clickable, not that they're both cooking videos. Similarity is a content question, so diversity uses the vectors. Real embeddings have hundreds of dimensions; the 2-D versions below are simplified so the cosine math is checkable by hand (read dim 1 as "cooking-ness," dim 2 as "music-ness").

Clarification — content embedding vs Two-Tower embedding, and which to use

Two kinds of video embedding exist, trained on different objectives. The **content embedding** (from the content encoder: thumbnail, audio, title) puts videos close when they're *about the same thing* — it exists the moment a video is uploaded. The **Two-Tower retrieval embedding** takes content features as input but is *trained on engagement*, so "close" means *engaged-with by the same crowd* — a cooking video and a football video loved by the same demographic land close in retrieval space but far apart in content space. For the diversity metric, both are available (the videos in the final list already cleared the funnel, so cold-start is irrelevant here — it matters upstream at retrieval, not at output-diversity). The Two-Tower vector is the convenient, already-cached choice and fine in practice; the content embedding is the refinement when you specifically want *topical* rather than *audience* diversity. The literature's verdict: both are standard, neither is universally right — state which you're using, because they measure different things. (One caution: embeddings change every retrain; compute a list's vectors from the same model snapshot, and never compare across versions.)

cos(a, b) = $(a \cdot b) / (\|a\| \cdot \|b\|)$, $a \cdot b = a_1b_1 + a_2b_2$, $\|a\| = \sqrt{(a_1^2 + a_2^2)}$

List diversity = average cos over all pairs — for 3 videos: [cos(A,B) + cos(A,C) + cos(B,C)] / 3

WORKED EXAMPLE — VERIFIED

Diverse list — A = (1, 0), B = (0.6, 0.8), C = (0, 1). All unit length — note ‖B‖ = √(0.6² + 0.8²) = √(0.36 + 0.64) = √1 = 1, the 3-4-5 triangle scaled down — so cosine reduces to the plain dot product:

- cos(A,B) = 1·0.6 + 0·0.8 = **0.6** · cos(A,C) = **0** · cos(B,C) = 0.6·0 + 0.8·1 = **0.8**
- Diversity = (0.6 + 0 + 0.8)/3 = **0.47** → low → diverse ✓

Redundant list — three videos all ≈ (1, 0): each pair ≈ 0.98 → average ≈ **0.98** → redundant ✓. ("Unit length" means the *squares* sum to 1, not the components; non-normalized vectors need the full ‖a‖·‖b‖ denominator.)

- **Diversify across creators too**, not just topics — one creator shouldn't dominate the slate even if each video is relevant.
- **Never use diversity alone** — a diverse list of irrelevant videos is useless; read it alongside NDCG/precision.

A3 · The per-stage trap, and the catch that motivates Part B

The per-stage trap: evaluating candidate-generator recall only against items the generator *surfaced* is circular — you can never detect the good items it failed to surface. Fix: evaluate against **unbiased inputs** — candidates from outside its normal window (beyond the ~10k it surfaces) or outside its scope entirely.

The catch: every offline metric is computed on data logged by the *current* system, so it rewards a new model for agreeing with what the current system already did. If a new model would surface a great video the old system never showed, there's no logged click for it — offline it looks like a *miss*, not a win. Offline metrics measure "do you reproduce the logged outcomes," not "do you make users happier." That gap is why you can't ship on offline numbers alone — which is Part B.

Part B — Online Evaluation

Offline metrics are scored on the current system's logs, so they reward agreement with what's already shown, not real user happiness. Online evaluation measures live behavior.

B1 · What A/B testing is — from scratch

You've built a new ranking model and think it beats production. You can't show both models to one person, and you can't give the new model to *everyone* (if it's secretly worse, you've hurt every user). So you run a controlled experiment, like a medical trial:

- **Randomly split** users into two groups.
- **Group A (control)** keeps the *old* model — the baseline.
- **Group B (treatment)** gets the *new* model.
- Run both **at the same time**, compare a metric (say CTR) between groups. B meaningfully higher → new model wins.

Three details make it trustworthy: **random assignment** — the groups end up identical in every other way (same mix of heavy/casual users, countries, times), so any metric difference is *caused* by the model and nothing else; **same time window** — both run concurrently, so a difference can't be "this week vs last week"; **stable buckets** — a user stays in one group for the whole test, so their experience is consistent.

B2 · The three metric groups

1. A/B Testing Metrics (engagement — the north star)

- **CTR** = (clicked videos) / (recommended videos). Insightful but can't separate a satisfying click from clickbait — never read alone.
- **Number of completed videos**, **total watch time**, **session watch time**
- **Return visit rate** — the best proxy for long-term satisfaction and the hardest to game (clickbait lifts CTR but kills return rate)
- **Long-term engagement trends**
- **Creator satisfaction** — is the engagement win just concentrating views on a few big creators?

2. User Experience Metrics (guardrails — must not regress)

- **Recommendation acceptance rate** — of what was shown, how much the user acted on
- **Survey feedback** — direct in-app "did this match your interests?" (the UTIS mechanism)
- **Negative feedback rate** — "not interested," hides, reports
- **Explicit user feedback** — videos explicitly liked/disliked; the cleanest direct read of opinion

3. System Health Metrics (is it shippable?)

- **Latency** — time to return recommendations, per request; hard constraint is the ~250 ms serving budget. Tracked at the **tail (p99)**, not the average. *p99 = the value 99% of requests come in under*; if p99 = 250 ms, 99% finished in ≤250 ms and the slowest 1% took longer. The average hides bad tails — 99 requests at 50 ms plus 1 at 5,000 ms averages to ~100 ms (looks fine), but one user waited 5 seconds. At a billion users the slowest 1% is 10 million people daily, so the budget is set against the tail: "even the unluckiest 1% stay under 250 ms."
- **Throughput** — requests handled per second; must survive peak traffic, or requests get dropped under spikes.

- **Resource utilization** — GPU/CPU/memory consumed to serve the traffic; a 3% watch-time gain that doubles the GPU fleet may not be worth it — the dollars-per-improvement check.
- **Error rate** — fraction of requests that fail (timeouts, crashes, empty results).

These are non-negotiable constraints, not things you trade against engagement: a model that lifts watch time 3% but blows the latency budget, can't handle peak load, triples cost, or errors out fails regardless of its Tier-1 numbers.

B3 · Is the difference real? — statistical significance

Supplied, not sourced: the sources name A/B testing but not significance testing; the z-test below is standard statistics, and the CTR counts in the example are illustrative (only the +5.2% relative lift is the real Reels number).

The catch. Control comes out 3.00% CTR, treatment 3.16%. Treatment looks better — but two *identical* models wouldn't give exactly equal CTRs either: users are random, and group B might just be slightly clickier by luck. So the real question is: *if the two models were actually equally good, how likely is luck alone to produce a gap this big?* Unlikely → the gap is real (**statistically significant**). Easily explained by luck → don't trust it.

Coin intuition: 6 heads in 10 flips proves nothing; 6,000 in 10,000 proves bias. Same 60%, more flips = more trust. A/B tests are identical — a CTR gap means little on 1,000 users and a lot on a million. Volume is what turns a delta into proof.

> **$z = (CTR_t - CTR_c) / SE$** — "how many times bigger is my gap than the random wobble?"
> **$SE = \sqrt{p(1-p) \cdot (1/n_c + 1/n_t)}$** $|z| > 1.96 \leftrightarrow$ p-value $< 0.05 \rightarrow$ significant.

The SE terms, written out: p = pooled CTR (both groups combined) ≈ 0.0308; **$p(1-p)$** = 0.0308 × 0.9692 = 0.02985 — the variance of a single yes/no click outcome (a click is a coin flip with probability p); **$1/n_c + 1/n_t$** — combines the uncertainty of both groups; more users in either shrinks it. In words: SE = √(per-user click variance × combined 1/sample-sizes).

WORKED EXAMPLE — SAME GAP, TWO SAMPLE SIZES, OPPOSITE VERDICTS (VERIFIED)

Gap = 3.158% − 3.00% = 0.158 points (+5.2% relative — the real Reels lift).

```
n = 1,000,000/group:  1/n_c + 1/n_t = 0.000002
  SE = √(0.02985 × 0.000002) = 0.000244  →  z = 0.00158/0.000244 ≈ 6.5  ≫ 1.96  → REAL

n = 1,000/group:      1/n_c + 1/n_t = 0.002
  SE = √(0.02985 × 0.002)    = 0.0077    →  z = 0.00158/0.0077  ≈ 0.2          → NOISE
```

Shrinking the sample 1,000× grew $(1/n_c+1/n_t)$ 1,000×, and the square root turned that into a √1000 ≈ 32× larger SE (0.000244 → 0.0077). With only 1,000 users a side, CTR naturally wobbles by roughly ±0.77 points from luck alone — a 0.158-point gap is one-fifth of that everyday noise. **Identical improvement, opposite conclusion; only the sample size changed.** Size the experiment before trusting any delta.

Clarification — can the numerator (and z) be negative?

Yes, and the sign carries meaning: $z > 0$ → treatment beat control; $z < 0$ → treatment *lost*; $z \approx 0$ → no real difference. Magnitude is strength of evidence — $z = -6.5$ is just as significant as $+6.5$, but it means you've confidently shown the new model is *worse* and must not ship. That's the test working, not failing: catching a regression before launch is what it's for. Hence the threshold is written $|z| > 1.96$ (two-sided — flags differences in either direction); a one-sided test ($z > 1.645$, "is treatment specifically better?") exists, but two-sided is the safer default because you want to know about harm, not just help.

B4 · The trap — novelty effect

Experiments often show inflated results because the new system is *novel*: if the old system served you A-type videos until you tired of them, the new system's B-type videos get a temporary bump that fades once B also gets stale. Mitigation: run long enough for novelty to wash out, and check whether the lift *decays* across the window rather than trusting day-one numbers.

B5 · Tuning the value-model weights

A separate activity from A/B testing: A/B asks "is model X better than model Y?"; tuning asks "what weight values are best?" — a search, not a head-to-head. The score is $VM = \Sigma\, W_k \cdot P(\text{head}_k)$ with hundreds of tunable knobs. Two approaches (Instagram Explore):

- **Bayesian optimization** — tune weights live: specify the goal metric plus regression thresholds on guardrails, let BO search. Trustworthy but slow — convergence can exceed a month with many low-sensitivity metrics.
- **Offline tuning** — learn a function mapping *offline* metric changes → *online* metric changes, then try weight settings offline in hours. Fast, but only valid if **offline and online metrics are strongly correlated** — and that correlation is the premise of Part C.

B6 · A real result, end to end — Reels UTIS

A clean template for how a launch is reported. Offline first (accuracy 59.5% → 71.5%, precision 48.3% → 63.2%, recall 45.4% → 66.1%), *then* an online A/B over **10M+ users** — a sample so large that even small deltas clear the significance bar comfortably:

metric	delta	tier
High survey ratings	+5.4%	engagement / satisfaction (Tier 1)
Low survey ratings	−6.84%	fewer "bad match" responses
Total engagement	+5.2%	Tier 1
Integrity violations	−0.34%	guardrail moved the right way (Tier 2)

Two lessons: **(1) offline → online order** — offline (cheap) screened the model, the A/B (expensive, trustworthy) confirmed it; **(2) the shape matters more than any single number** — engagement up *and* the integrity guardrail improved. The same +5.2% with integrity *rising* would be a fail, however good the CTR — which is the entire reason guardrails are tracked alongside the north star.

Part C — Debiasing

The problem the whole section has been circling

Three facts from Parts A and B: offline metrics reward a new model for *mimicking* what the current system showed (A's catch); online A/B tests measure the truth but are slow, expensive, and risky (B); offline tuning only works if offline and online metrics correlate (B5). Debiasing removes the systematic distortions in logged data so the cheap offline estimate becomes a trustworthy predictor of the expensive online result. The sentence that names the enemy (Hello Interview): *without debiasing, a newly trained model eagerly learns to approximate the output of the current system instead of the true user preferences.*

Where the bias comes from

Training labels aren't "what users like" — they're "what users did, **given what the current system chose to show them**." That conditioning injects:

- **Position bias** — a video clicked at slot 1 isn't necessarily better than one ignored at slot 10; the top slot just gets *seen* more.
- **Popularity bias** ("rich get richer") — already-popular videos get shown more → more clicks → look better → shown even more. A feedback loop.
- **Exposure / selection bias** — no label at all for videos the system never showed; the good video the old system missed is invisible in the logs.

Fix 1 — Position bias correction (in the loss)

Where it lives: training only — a term in the heavy ranker's objective ($L = L_{engage} + L_{aux} + L_{position}$); once training ends it's gone, and the model simply *is* less position-fooled.

> $L_{position} = \delta \cdot BCE(y\_true, y\_pred) \cdot$ **position_weight** — down-weights clicks from high-visibility slots so the model doesn't mistake "shown at the top" for "good."

EXAMPLE

Two clicked videos. A click at slot 1 is *expected* (the slot gets all the attention) → low position_weight (0.3), contributes 0.3 to the loss. A click at slot 10 is *surprising* → full weight 1.0. The model learns more from the surprising click — taught quality, not visibility.

Fix 2 — Inverse-Propensity Weighting (IPW)

Where it lives: two places — training and offline evaluation. It's not a metric; it's a sample reweighting. In *training*, each example's contribution to the heavy ranker's loss is multiplied by 1/propensity — not a new loss term, but a coefficient wrapped around each example's existing loss: $L = \Sigma\ (1/propensity_i) \cdot loss_i$. In *offline evaluation*, the same weights are applied inside the metric (weighted CTR, weighted NDCG), so the offline number estimates performance under *fair* exposure — off-policy estimation, the use that makes offline predictive of online.

> **Propensity** = P(video was examined | its placement) — top slot ≈ 0.9, mid ≈ 0.3. Each logged example is weighted by **1/propensity**: observations that happened *despite* low exposure count for more. (A slot seen 30% of the time means each click there stands in for ~3 clicks' worth of full-exposure behavior.)

Clarification — IPW vs position-bias correction: aren't they the same?

They share the goal but differ in scope. **$L_{position}$ = position-only, training-only**: a term welded inside each example's loss. **IPW = any exposure bias, in training and evaluation**: a general principle — reweight by 1/exposure-probability — where the exposure probability can be built around position, popularity, or which generator served the item, and which also works in offline evaluation (no $L_{position}$ equivalent exists there). $L_{position}$ is essentially the position-only, in-loss special case of the IPW idea: one is a summand, the other a coefficient. A system can run both: $L_{position}$ as the cheap built-in fix during training, IPW separately for debiased offline metrics and non-position biases.

Clarification — who decides the propensity values?

Nobody sets them; they're *estimated*, and that estimation is the hard part. Three standard sources: **(1) a position/examination model** — e.g. the same item appears at different positions over time, and the ratio of its click rates across slots reveals the slots' relative examination probabilities, independent of the item's quality; **(2) the logging policy itself** — a stochastic ranker knows the probability it served each item and stamps it on the log at serving time (the principled version, and why counterfactual logging pairs with IPW); **(3) randomized data** — the uniform-sampled refresh (Fix 4) has exposure that's random by construction, so true examination probabilities are known there; you fit or validate the propensity model on that clean slice, then apply it to the biased bulk. An engineer chooses the method and the variables (position only? + device? + popularity?); the values come from data.

WORKED EXAMPLE, END TO END — VERIFIED

Setup: videos A and B are *equally good* — true click-if-examined probability 0.40 for both. The old system always showed A at slot 1, B at slot 8. Estimated propensities: slot 1 examined 80% of the time (0.80), slot 8 examined 20% (0.20). Each video shown 1,000 times; clicks = quality × P(examined) × shows.

```
BIASED LOGS:    A @ slot1: 0.40 × 0.80 × 1000 = 320 clicks
                B @ slot8: 0.40 × 0.20 × 1000 =  80 clicks
                naive read: A looks 4.0× better  ← WRONG (pure position artifact)

IPW-CORRECTED:  A: 320 × 1/0.80 = 320 × 1.25 = 400
                B:  80 × 1/0.20 =  80 × 5.00 = 400
                corrected: A = B = 400  → truth recovered
```

B's clicks each counted 5× because they were earned under 5× worse exposure — exactly cancelling A's visibility advantage.

The blowup and the fix: a slot examined only 2% of the time gives weight 1/0.02 = 50 — one lucky click counts as 50 and a single noisy observation can dominate the loss or metric (high variance). **Clipping** caps the weight (say at 10) so rare-exposure examples still count extra but can't hijack the result; **self-normalized IPW** (divide by the sum of weights) is the other standard guard. IPW is only as good as its propensity estimates.

Fix 3 — Counterfactual logging

Where it lives: serving/logging — upstream of models and metrics entirely. A data-collection policy: a change to what the serving system *records*. Its output is data, which training and evaluation then consume. It attacks exposure bias — the missing labels for videos never shown — by logging what feedback a video *would* have gotten under a different policy.

EXAMPLE

A new candidate generator wants to surface video Z, but the live system never showed Z — the logs have zero signal on it, so offline the new generator gets penalized for disagreeing with the old one. With counterfactual logging you record "Z would have been served to user U here" (ideally injecting it to a tiny holdout for a real outcome), so the offline evaluator can credit the new generator instead — escaping the "logs only contain what we already chose" trap.

Fix 4 — Periodic uniform-sampled refreshes

Where it lives: serving — a small slice of live traffic. Its product is data used downstream in three places: training (debiased examples), evaluation (true candidate-generator recall — Part A's "unbiased inputs"), and monitoring (popularity-drift detection). It's also where the propensities IPW needs get estimated — the fixes feed each other. Serve a small fraction of slates with videos chosen **uniformly at random**; random exposure means those labels carry no position or popularity bias.

WORKED EXAMPLE, SLOWLY — VERIFIED

The trap raw counts set: "got clicked a lot" mixes up two different things — "is this video good?" and "was this video shown a lot?" Two videos, *equally good*: when a user actually sees either, they click 20% of the time (true per-view CTR = 0.20). The biased system showed the popular one 1,000× and the niche one 100×:

```
BIASED LOGS:     popular: 1000 × 0.20 = 200 clicks
                 niche:    100 × 0.20 =  20 clicks
                 raw counts: popular "10× better"  ← lie: shown 10× more, same quality

UNIFORM REFRESH: show each 500× (randomly, not by the model)
                 popular: 500 × 0.20 = ~100 clicks
                 niche:   500 × 0.20 = ~100 clicks   → gap evaporates, equal quality exposed
```

By forcing equal random exposure you remove exposure as a variable — whatever click difference remains must be real quality, because it can't be exposure anymore. If you'd trained on the biased logs instead, the model "learns" the popular video is great, shows it more, it gets more clicks, looks greater — the rich get richer on no real merit.

Clarification — this only happens on the random slice; how does a tiny slice help?

The slice is tiny by design (you can't serve random content at scale), and it never needs to be the training set — its jobs are *leverage* jobs. **(1) A measurement ruler:** on the clean slice you know true per-view quality, so you can audit the biased system ("normal logs say X is 10× better; the random slice says equal → we're over-promoting on exposure") and validate candidate-generator recall honestly — a small unbiased sample can measure a bias even when it's too small to retrain on, the way a few thousand poll respondents estimate a national election. **(2) A propensity calibrator:** you can't estimate P(examined | position) from biased logs alone — the bias is what you're removing; on the randomly-exposed slice exposure is known by construction, so you fit the propensity model there and apply it (via IPW) to the *entire* biased bulk. The small slice teaches the correction; the correction scrubs the large dataset. **(3) An exploration valve:** the popularity loop starves niche/new videos of any exposure, so they can never prove themselves; the random slice guarantees every video occasionally gets shown, breaking rich-get-richer at the source.

Clarification — is the random slice representative? Different random slices differ, right?

Both true, and they're the two halves of sampling. **Unbiased:** random assignment means no systematic skew — on average the slice has the same mix of users, videos, positions as the population; that's what makes it a valid ruler. **Variance:** any single draw wobbles around that average, like two polls of 1,000 people giving slightly different numbers — and smaller slices wobble more (the same $1/\sqrt{n}$ story as the significance section). Consequences: size the slice for the *rarest* thing you need to estimate (rare-position propensities need enough random impressions landing there, or the estimate is noisy — and a noisy propensity in a 1/propensity denominator is exactly the blowup); aggregate over a rolling window so variance averages down; and report confidence intervals, not falsely precise point estimates.

Why this closes the loop

The first two fixes correct how biased data is *used* ($L_{position}$ and IPW, in training/eval); the last two create *unbiased data* at the source (counterfactual logging and uniform refreshes, in serving/logging). Each makes the offline data look more like *what users would do under fair exposure* rather than *what they did under the current system's biases*. The better that holds, the stronger the offline ↔ online correlation — and the more you can trust offline tuning to predict A/B outcomes, killing bad models before paying for the live test. That's the arc of Section 5: offline (cheap, biased) → online (true, costly) → debiasing (make cheap predict costly).
